🔍 Product Search - GraphQL API

Additional Resources:

- Product Search Fields - GraphQL API

Core Idea

When a user searches for a product, the system applies a cascade of search strategies — moving from the most precise to the most forgiving.
As soon as one strategy produces results, the process stops. If it returns no hits, the system falls through to the next strategy.

This ensures that exact or near-exact matches are preferred, while still returning useful results if the user input is incomplete or slightly incorrect.

Summary Table

Strategy	Precision	Typo-Tolerant	Partial Input
1 – Begins-With	Medium	No (except compound splits)	Yes
2 – Begins-With + Typo-Tolerant	Lowest	Yes	Yes

The system always attempts the most precise interpretation first and becomes more forgiving only when necessary — maximizing relevance while minimizing cases where no results are returned.

The Strategies (in order)

1. Begins-With Match

The system first tries to search for all entered words as a prefix.
For longer terms (more than 7 characters), it also recognizes common German compound-word variations — whether a term is written together, split with a space, or hyphenated.

Example:

- `Grippe` → matches `Grippeimpfstoff`, `Grippemittel`, etc.
- `aspirin` → matches `Aspirin`
- `aspi` → matches `Aspirin`
- When searching `Grippeimpfstoff`, `Grippe Impfstoff`, or `Grippe-Impfstoff`, the system recognizes all variations as equivalent and correctly returns results for `Grippe-Impfstoff`.

When this helps:
Users enter the correct term or the beginning of the term.
Due to German compound-word flexibility, they may write the entire word together, separately, or with a hyphen. The system bridges these variations to ensure the correct product is still found.

---

3. Begins-With + Typo-Tolerant Match

As a last fallback, the system adds typo-tolerance on top of prefix matching.
Slight misspellings will still match relevant products.
Typo-tolerant search is only applied to segments of more than 4 characters

Example:

- `Grippe` → matches `Grippeimpfstoff`, `Grippemittel`, etc.
- `Gripe` → matches `Grippe`. But does not match `Grippeimpfstoff`, `Grippemittel`, etc.

When this helps:
Users make spelling mistakes or are unsure of the exact wording.

Advanced search options

Special operators

When a user includes advanced search operators in their input — such as a minus sign (`-`) to exclude terms, an asterisk (`*`) for wildcard matching, or a tilde (`~`, `~1`, `~2`) for fuzzy/typo-tolerant/proximity-control — the system recognizes these as intentional, expert-level instructions. Rather than modifying or re-interpreting such terms through its usual relaxation strategies, it preserves them exactly as the user typed them. This ensures that power users who deliberately craft precise search expressions are not undermined by the automatic fallback logic, while casual users who type plain words continue to benefit from the progressive matching strategies.

`?` **single-character wildcard**

`asp??in` finds `aspirin` because the two question marks match `ir`

`*` **multi-character wildcard**

`asp*in` finds `aspirin` because `*` matches `ir`

`-` **discard if found**

`aspirin -N02BA51` finds all `aspirin` entries, which do not have ATC code `N02BA51`

Quoted Phrases

If a user surrounds part of the query with quotes, e.g. `"grippe impfstoff"`, that segment is treated as a **fixed, exact phrase** across all three strategies.

Only the *unquoted* search terms go through relaxation (compound split tolerance, prefix logic, typo tolerance).